Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Electrónica

Diseño de Sistemas Digitales (EL-3310)

Profesor: Ing. Michael González Vega

Informe Técnico Corto

Capítulo 1: Los Cimientos

Diseño e Implementación de Componentes Base para un Procesador RISC-V Uniciclo

Integrantes

Ximena Araya Brenes (Carné: 2024122095) — **Team Lead**

Kevin Méndez Carvajal (Carné: 2023328177) — **Hardware Developer**

John Hibbert Bonilla (Carné: 2024062788) — **Verification Engineer**

Jason Ulloa Picado (Carné: 2024159001) — **Documentation & Configuration Engineer**

1. Descripción General de los Componentes

En esta primera etapa del proyecto final, se diseñaron, implementaron y verificaron funcionalmente los módulos digitales en SystemVerilog. Los componentes desarrollados son:

- **Unidad Lógica Aritmética (`alu.sv`):** Módulo combinacional de 32 bits encargado de las operaciones aritméticas y lógicas (`ADD`, `SUB`, `AND`, `OR`), generando la bandera `Zero` cuando el resultado es igual a cero.
- **Banco de Registros (`register_file.sv`):** Arreglo de 32 registros de 32 bits con dos puertos de lectura asíncrona y un puerto de escritura síncrona habilitado por la señal `RegWrite`.
- **Generador de Inmediatos (`imm_gen.sv`):** Módulo encargado de extraer y reconstruir los inmediatos que vienen de las instrucciones, haciendo la extensión de signo a 32 bits para formatos I-type y S-type.
- **Multiplexor 2 a 1 (`mux.sv / mux2to1`):** Selector parametrizable (`WIDTH = 32`) diseñado para el enrutamiento y selección de datos dentro del datapath.

2. Decisiones Principales de Diseño

2.1. Unidad Lógica Aritmética (ALU)

La ALU fue implementada mediante un bloque combinacional `always_comb` que evalúa el código de operación `ALUControl` de 4 bits. La señal `Zero` se genera comparando si el resultado evaluado es exactamente igual a cero:

```
if (Result == 32'b0)
    Zero = 1'b1;
else
    Zero = 1'b0;
```

2.2. Banco de Registros

Para asegurar que el registro $x0$ siempre esté en cero sin importar los intentos de escritura, se añadió una condición de validación en la lógica síncrona controlada por `RegWrite` y la dirección `A3`:

```
always_ff @(posedge clk) begin
    if (RegWrite && (A3 != 5'd0))
        registros[A3] <= WD3;
end
```

Las lecturas en los puertos `RD1` y `RD2` son asíncronas e independientes, retornando `32'd0` si se acciona la dirección `5'd0`.

2.3. Generador de Inmediatos

Se implementó un multiplexor interno controlado por la señal `ImmSrc` sobre el bus de instrucción `Instr`:

- **ImmSrc = 00 (I-type):** Concatenación y extensión de signo de {{20{`Instr[31]`}}, `Instr[31:20]`}.
- **ImmSrc = 01 (S-type):** Concatenación de campos y extensión de signo de {{20{`Instr[31]`}}, `Instr[31:25], Instr[11:7]`}.

2.4. Multiplexor 2 a 1 (`mux2to1`)

El módulo `mux2to1` se parametrizó por defecto a 32 bits y utiliza una asignación combinacional continua mediante operador ternario:

```
assign Y = Select ? B : A;
```

3. Verificación

Cada componente posee un testbench autocontenido (`*_tb.sv`) que usa estímulos de prueba y valida de forma automática el valor esperado contra el obtenido, imprimiendo en la consola de EDA Playground el estado de `PASS` o `FAIL`.

4. Resultados Obtenidos y Matriz de Pruebas

La matriz de pruebas ejecutada para la comprobación funcional de todos los módulos desarrollados es:

ID	Módulo	Entrada / Condición	Resultado Esperado	Estado
ALU-01	ALU	ADD: A=5, B=3, Ctrl=0000	Result=8, Zero=0	PASS
ALU-02	ALU	SUB: A=5, B=3, Ctrl=0001	Result=2, Zero=0	PASS
ALU-03	ALU	AND: A=5, B=3, Ctrl=0010	Result=1, Zero=0	PASS
ALU-04	ALU	OR: A=5, B=3, Ctrl=0011	Result=7, Zero=0	PASS
ALU-05	ALU	ZERO: A=5, B=5, Ctrl=0001	Result=0, Zero=1	PASS
ALU-06	ALU	NEGATIVE: A=3, B=5, Ctrl=0001	Result=-2 (0xFFFFFFFE)	PASS
ALU-07	ALU	Falla intencional: A=5, B=3	Result=9 (Inyección error)	FAIL (Esperado)
RF-01	Register File	Write x1=25; Read x1	RD1=25	PASS
RF-02	Register File	Read x1 y x2 simultáneo	RD1=40, RD2=60	PASS
RF-03	Register File	Write x0=100; Read x0	RD1=0	PASS
RF-04	Register File	RegWrite=0; Write x3=99	RD1=75 (Sin cambio)	PASS
IMM-01	Imm Gen	I-type (+): Instr (Imm=5), ImmSrc=00	ImmExt=0x00000005	PASS
IMM-02	Imm Gen	I-type (-): Instr (Imm=-1), ImmSrc=00	ImmExt=0xFFFFFFFF	PASS
IMM-03	Imm Gen	S-type (+): Instr (Imm=0), ImmSrc=01	ImmExt=0x00000000	PASS
IMM-04	Imm Gen	S-type (-): Instr (Imm=-4), ImmSrc=01	ImmExt=0xFFFFFFFC	PASS
IMM-05	Imm Gen	S-type (+): Instr (Imm=32), ImmSrc=01	ImmExt=0x00000020	PASS
MUX-01	MUX	Select Entrada A: Select = 0	Y = 0x12345678	PASS
MUX-02	MUX	Select Entrada B: Select = 1	Y = 0xABCDEF00	PASS
MUX-03A	MUX	Alt A: Select = 0	Y = 0x0000000A	PASS
MUX-03B	MUX	Alt B: Select = 1	Y = 0x00000014	PASS

Cuadro 1: Matriz de pruebas funcional basada en las ejecuciones de EDA Playground.

5. Problemas Encontrados y Soluciones

- **Escritura involuntaria en la dirección $x0$:** Inicialmente, los valores dirigidos a $x0$ modificaban la posición 0 del arreglo. Se solucionó añadiendo la condición de guarda `A3 != 5'd0` en la habilitación de escritura.
- **Advertencias de compilación en EDA Playground:** Con Icarus Verilog se generaron avisos sobre selecciones constantes dentro de bloques combinacionales (*constant selects in always_* processes*). Se comprobó que eran avisos no críticos del sintetizador y que la extensión de signo se realizaba correctamente.

6. Conclusiones

- Se diseñaron e implementaron con éxito los cuatro módulos base (`alu`, `register_file`, `imm_gen` y `mux2to1`) en SystemVerilog.
- La verificación automática en los testbenches permitió confirmar la correcta respuesta ante operaciones lógicas, aritméticas, almacenamiento de registros y extensiones de signo.
- Los cimientos del procesador se encuentran completamente probados, documentados y listos para integrarse en el datapath durante la siguiente fase.

Referencias

[1] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2.ª ed., Morgan Kaufmann, 2021.

[2] S. L. Harris y D. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, Morgan Kaufmann, 2021.

[3] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213, 2019. [En línea]. Disponible en: `https://riscv.org/technical/specifications/`

[4] IEEE Computer Society, *IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2017.

[5] D. Patterson y A. Waterman, *Guía práctica de RISC-V. El Atlas de una Arquitectura Abierta*, Strawberry Canyon LLC, 1.ª ed., 2017.

[6] M. González Vega, *Especificación del Proyecto Final: Capítulo 1 - Los Cimientos*, Curso EL-3310 Diseño de Sistemas Digitales, Instituto Tecnológico de Costa Rica, 2026.